

# คู่มือการใช้ระบบ

การเชื่อมต่อระบบ Webhook สำหรับรับข้อมูลการตรวจจับป้ายทะเบียนรถยนต์ (LPR) แบบเรียลไทม์

เอกสารฉบับนี้อธิบายรูปแบบและวิธีการเชื่อมต่อระบบ Webhook สำหรับรับข้อมูลการตรวจจับป้ายทะเบียนรถยนต์ (LPR) แบบเรียลไทม์ เมื่อมีรถยนต์วิ่งผ่านจุดตรวจ ระบบของเราจะทำการส่งข้อมูล (HTTP POST) ไปยัง Endpoint URL ที่ท่านได้ลงทะเบียนไว้ในระบบโดยอัตโนมัติ

## 1. ข้อมูลการเชื่อมต่อ

ระบบของเราจะส่งข้อมูลไปยังเซิร์ฟเวอร์ของท่านด้วยรูปแบบดังนี้:

|              |                               |
|--------------|-------------------------------|
| HTTP Method  | POST                          |
| Content-Type | multipart/form-data           |
| Timeout      | สูงสุด 10 วินาที ต่อ 1 คำสั่ง |

หมายเหตุ: 10 วินาที คือ Timeout ตอนส่งข้อมูลป้ายทะเบียนจริง ส่วนการยิงทดสอบ (Dummy Post ตอนข้อ 2 และการทดสอบสถานะซ้ำทุก 30 นาที ตอนข้อ 7) ระบบจะใช้ Timeout สั้นกว่าคือ 5 วินาที

## 2. การส่ง Dummy Post เพื่อทดสอบ URL

User ต้องทำการกรอก URL Endpoint ของตัวเองก่อนและต้องมี schema เป็น https เท่านั้น

ระบบจะทำการยิง Dummy Post ไปทาง URL ของท่านเพื่อทดสอบว่าปลายทางสามารถรับข้อมูลได้จริงโดยมีรูปแบบดังนี้

| ชื่อฟิลด์ (Field) | คำอธิบาย                                  | ตัวอย่างข้อมูล      |
|-------------------|-------------------------------------------|---------------------|
| Camera_id         | รหัสอ้างอิง id ของกล้อง(ข้อมูลจำลอง)      | TEST_Camera_0001    |
| event_id          | รหัสอ้างอิงเหตุการณ์ตรวจจับ (ข้อมูลจำลอง) | TEST_Event_0001     |
| license_plate     | หมายเลขทะเบียนรถ (ข้อมูลจำลอง)            | กข 1234             |
| province          | จังหวัดของป้ายทะเบียน (ข้อมูลจำลอง)       | กรุงเทพมหานคร       |
| color             | สีของตัวรถ (ข้อมูลจำลอง)                  | White               |
| capture_time      | วันที่และเวลา (ข้อมูลจำลอง)               | 2026/07/09 14:30:00 |
| image_crop        | ไฟล์รูปตัวอย่าง                           | ไฟล์รูปภาพ (JPEG)   |

| ชื่อฟิลด์ (Field) | คำอธิบาย                             | ตัวอย่างข้อมูล    |
|-------------------|--------------------------------------|-------------------|
| Camera_id         | รหัสอ้างอิง id ของกล้อง(ข้อมูลจำลอง) | TEST_Camera_0001  |
| image_full        | ไฟล์รูปตัวอย่าง                      | ไฟล์รูปภาพ (JPEG) |

เซิร์ฟเวอร์ของท่านต้องตอบกลับตามเงื่อนไขดังต่อไปนี้:

|                  |                        |
|------------------|------------------------|
| HTTP Status Code | ต้องตอบกลับเป็น 2XX OK |
| Content-Type     | application/json       |

รูปแบบ **JSON**: ต้องส่ง **event\_id** กลับมายืนยันตามโครงสร้างด้านล่างนี้

ตัวอย่าง JSON ที่เซิร์ฟเวอร์ของท่านต้องตอบกลับ:

```
{
  "event_id": "TEST_Event_id_0001"
}
```

### 3. การเพิ่มกล้องและลบกล้อง

3.1) User ต้องทำการยิงข้อมูลรูปแบบ Jason มาที่ Endpoint ของทางระบบ :

**[BASE\_URL]/partner/cameras (POST)**

โดยผู้ใช้ต้องเข้าไปเพื่อขอ API Key จากระบบของเราและแนบมาทาง Header เพื่อใช้ยืนยัน

ตัวอย่าง : sk\_live\_XXXXXXXXXXXXXXXXXX

\*กล้อง1ตัวสามารถเพิ่มได้แค่ 1 url Endpoint เท่านั้น

\*webhook\_url ที่ระบุ ต้องเป็น URL ที่ท่านลงทะเบียนไว้แล้วล่วงหน้าผ่านหน้าเว็บ (เมนู Webhook Endpoint) ก่อนเท่านั้น จะพิมพ์ URL ใหม่ที่ยังไม่เคยลงทะเบียนไม่ได้

\*จำกัดการเพิ่มกล้องไม่เกิน 20 ครั้ง ต่อ 1 ชั่วโมง ต่อบัญชี

\*หากบัญชีของท่านถูกผู้ดูแลระบบระงับการใช้งานชั่วคราว API Key ของท่านจะไม่สามารถเรียกใช้งาน Endpoint ใดๆ ในหมวด Partner Integration นี้ได้เลย (เพิ่มกล้อง, เปิด/ปิดกล้อง, เช็คสถานะกล้อง, ลบกล้อง) ระบบจะตอบกลับ 403 Forbidden จนกว่าจะได้รับการปลดระงับ

\*User ต้องส่งเวลา delay ในการจับป้ายทะเบียนมาด้วย (เมื่อกล้องจับป้ายทะเบียนได้จะ delay ไม่จับป้ายทะเบียนซ้ำตามเวลาที่ user กำหนด จำกัดเวลา 1 ถึง 300 วินาที แล้วจะเริ่มจับใหม่อีกครั้ง)

ตัวอย่าง JSON ที่ User ต้องส่งมาที่ Endpoint ของทางระบบ:

```
{
  "camera_id": "lsdjcmxzncjelacmldcd",
  "camera_url": "rtsp://cityapp.planetcloud.cloud:8554/LPR-Test-01",
  "webhook_url": "https://asdokasod.com/",
  "delay": 5
}
```

**\*\*ถ้าระบบของ User ส่งมาไม่ถูกต้องตามรูปแบบที่กำหนดไว้หรือลิงค์กล้องไม่สามารถใช้งานได้ระบบจะตอบ Error กลับไป\*\***

ในการเช็คสถานะกล้องว่ากล้องตัวนั้นอยู่สถานะอะไร User ต้องทำการ Poll ข้อมูลโดยแนบ header มาที่ Endpoint ของทางระบบ : **[BASE\_URL]/partner/cameras/{camera\_id}** (GET)

โดยต้องแนบ Header X-API-Key ไปเดียวกับที่ใช้ตอนเพิ่มกล้อง (ขอได้จากเมนู API Key ในเว็บของท่าน) เพื่อยืนยันตัวตนทุกครั้งที่เราเรียก Endpoint นี้

**\*จำกัดการเรียกเช็คสถานะไม่เกิน 2,400 ครั้ง ต่อ 1 ชั่วโมง ต่อบัญชี**

**รูปแบบข้อมูลที่ตอบกลับ (Response Body):**

| ชื่อฟิลด์ (Field)   | คำอธิบาย                                                                                 | ตัวอย่างข้อมูล     |
|---------------------|------------------------------------------------------------------------------------------|--------------------|
| camera_id           | รหัสอ้างอิง id ของกล้องที่ต้องการเช็คสถานะ                                               | lsdjcmxznclacmldcd |
| verification_status | สถานะการตรวจสอบ RTSP ของกล้อง (pending / verified / failed) ดูความหมายแต่ละสถานะด้านล่าง | verified           |
| is_active           | สถานะเปิด/ปิดใช้งานกล้องในปัจจุบัน                                                       | true               |

ความหมายของสถานะ **verification\_status** แต่ละแบบ:

- **pending** — กำลังรอตรวจสอบ ระบบจะทดลองเชื่อมต่อ RTSP ให้อัตโนมัติทุก 30 วินาที จนกว่าจะครบจำนวนครั้งที่กำหนด
- **verified** — เชื่อมต่อ RTSP สำเร็จแล้ว กล้องพร้อมใช้งาน
- **failed** — เชื่อมต่อ RTSP ไม่สำเร็จในรอบล่าสุด ระบบจะลองเชื่อมต่อใหม่ให้อัตโนมัติในรอบถัดไป

\*ระบบจะลองเชื่อมต่อ RTSP ให้สูงสุด 5 ครั้ง ถ้ายังไม่สำเร็จ ระบบจะลบกล้องตัวนั้นออกจากระบบโดยอัตโนมัติ (ต้องสร้างกล้องใหม่ผ่าน [BASE\_URL]/partner/cameras อีกครั้ง) หากเรียก Endpoint นี้ด้วย camera\_id ที่ถูกลบไปแล้ว หรือไม่เคยมีในระบบ ระบบจะตอบกลับ 404 Not Found

3.2) ในการลบกล้องออกจากระบบ **User** ต้องทำการยิง Request มาที่ Endpoint ของทางระบบ :

**[BASE\_URL]/partner/cameras/{camera\_id}** (DELETE)

โดยต้องแนบ Header X-API-Key ไปเดียวกับที่ใช้ตอนเพิ่มกล้อง (ขอได้จากเมนู API Key ในเว็บของท่าน) เพื่อยืนยันตัวตนทุกครั้งที่เราเรียก Endpoint นี้

**\*จำกัดการลบกล้องไม่เกิน 20 ครั้ง ต่อ 1 ชั่วโมง ต่อบัญชี**

**\*เมื่อลบกล้องสำเร็จ camera\_id และลิงก์ RTSP (camera\_url) ของกล้องตัวนี้จะสามารถนำไปใช้สร้างกล้องใหม่ได้ทันที \***

หากไม่พบกล้อง camera\_id ที่ระบุในบัญชีของท่าน ระบบจะตอบกลับ 404 Not Found

หมายเหตุ: Endpoint นี้ไม่มี Body เป็น JSON ที่ต้องส่ง — **camera\_id** ถูกส่งผ่าน URL Path โดยตรง และยืนยันตัวตนผ่าน Header **X-API-Key** เท่านั้น ตัวอย่างการเรียกใช้งานและรูปแบบข้อมูลที่ระบบตอบกลับมีดังนี้:

**ตัวอย่าง** การเรียกใช้งาน (Request):

```
DELETE [BASE_URL]/partner/cameras/lsdjcmxzncjelacmldcd
X-API-Key: sk_live_XXXXXXXXXXXXXXXXXX
```

**ตัวอย่าง JSON** ที่ระบบจะตอบกลับเมื่อลบกล้องสำเร็จ:

```
{
  "message": "ลบกล้อง 'lsdjcmxzncjelacmldcd' ออกจากระบบเรียบร้อยแล้ว "
}
```

**ตัวอย่าง JSON** ที่ระบบจะตอบกลับเมื่อไม่พบกล้อง (404 Not Found):

```
{
  "detail": "ไม่พบกล้อง camera_id='lsdjcmxzncjelacmldcd' ในบัญชีของคุณ"
}
```

## 4. รูปแบบข้อมูลที่ส่ง (Body ที่ส่งไป)

ข้อมูลจะถูกส่งไปในรูปแบบ **Form-Data** โดยประกอบด้วยข้อมูลตัวอักษรและไฟล์รูปภาพ ดังตารางด้านล่างนี้:

| ชื่อฟิลด์ (Field) | คำอธิบาย                                                           | ตัวอย่างข้อมูล                    |
|-------------------|--------------------------------------------------------------------|-----------------------------------|
| camera_id         | รหัสอ้างอิง id ของกล้อง                                            | Jdsdivnjcebjslsdapxjvncsjnd       |
| event_id          | รหัสอ้างอิงเหตุการณ์ตรวจจับ<br>ใช้สำหรับอ้างอิงและป้องกันข้อมูลซ้ำ | djfsjhjVCNSDkjhcNDNCjewzc         |
| license_plate     | หมายเลขทะเบียนรถที่ตรวจจับได้                                      | 1กข1234                           |
| province          | จังหวัดของป้ายทะเบียน                                              | กรุงเทพมหานคร                     |
| color             | สีของตัวรถที่จับได้                                                | Red (หรือ unknown ถ้าทายสีไม่ได้) |
| capture_time      | วันที่และเวลาที่ตรวจจับภาพได้                                      | 2026/07/09 14:30:00               |
| image_crop        | ไฟล์รูปภาพที่ตัดมาเฉพาะส่วนป้ายทะเบียนรถ                           | ไฟล์รูปภาพ (JPEG)                 |
| image_full        | ไฟล์รูปภาพรถยนต์แบบเต็มคัน ณ เวลาที่ตรวจจับได้                     | ไฟล์รูปภาพ (JPEG)                 |

หมายเหตุ: **capture\_time** ส่งมาเป็นรูปแบบ **YYYY/MM/DD HH:MM:SS** (เช่น 2026/07/09 14:30:00) เป็น string ธรรมดา ไม่ใช่ ISO 8601 กรุณาเขียนโค้ด parse ให้ตรงรูปแบบนี้โดยเฉพาะ

## 5. รูปแบบการตอบกลับที่ต้องการ

เพื่อให้ระบบของเราทราบว่าเซิร์ฟเวอร์ของท่านได้รับข้อมูลอย่างสมบูรณ์แล้ว

เซิร์ฟเวอร์ของท่านต้องตอบกลับตามเงื่อนไขดังต่อไปนี้:

|                         |                        |
|-------------------------|------------------------|
| <b>HTTP Status Code</b> | ต้องตอบกลับเป็น 2XX OK |
| <b>Content-Type</b>     | application/json       |

รูปแบบ **JSON**: ต้องส่ง **event\_id** กลับมายืนยันตามโครงสร้างด้านล่างนี้

ตัวอย่าง **JSON** ที่เซิร์ฟเวอร์ของท่านต้องตอบกลับ:

```
{
  "event_id": " djfshjVCNSDkjhcNDNCjewzc "
}
```

**\*\*User** ต้องตอบกลับตามรูปแบบนี้ทุกครั้งแม้จะได้รับข้อมูลซ้ำ เพื่อป้องกันในกรณีที่สัญญาณตกหล่นและทางระบบไม่ได้รับการตอบกลับของท่านในครั้งแรกที่ระบบส่งข้อมูลไป\*\*

## 6. การเปิดปิดกล้องการตั้งเวลาdelay

User ต้องทำการยิงข้อมูลรูปแบบ Jason มาที่ Endpoint ของทางระบบ :

**[BASE\_URL]/partner/cameras/status (POST)**

โดยผู้ใช้งานต้องเข้าไปเพื่อขอ API Key จากระบบของเราและแนบมาทางHeader เพื่อใช้ยืนยัน

**1.ตัวอย่าง JSON** ที่ User ต้องส่งมาที่ Endpoint ของทางระบบ:

```
{
  "camera_id": "lsdjcmxzncjelacmldcd",
  "is_active": true/false
}
```

**\*\*เมื่อทางระบบของUserส่งrequestมาเพื่อปิดกล้อง ทางระบบจะทำการหยุดส่งข้อมูลจากกล้องตัวนั้น ภายในไม่เกิน 30 วินาที (รอบตรวจสอบถัดไปของระบบ) \*\***

**\*\*การสั่ง is\_active: true (เปิดกล้อง) จะสำเร็จก็ต่อเมื่อกล้องตัวนั้นผ่านการตรวจสอบ RTSP จนมีสถานะ verified แล้วเท่านั้น ถ้ายังเป็น pending หรือ failed ระบบจะตอบ Error กลับไป**

### 1. ตัวอย่าง JSON ที่ User ต้องส่งมาที่ Endpoint ของทางระบบเพื่อตั้งเวลา delay:

```
{
  "camera_id": "lsdjcmxzncjelacmldcd",
  "delay": 120
}
```

**\*User สามารถส่งมาพร้อมกันทั้งสถานะการเปิดปิดหรือเวลา delay ได้เลยมาที่ Endpoint นี้ได้เลย\***

## 7. นโยบายเมื่อเกิดข้อผิดพลาด

หากระบบของเราส่งข้อมูลไปแล้วพบปัญหา เช่น:

- เซิร์ฟเวอร์ของท่านล่ม (ตอบกลับ 500 Internal Server Error)
- ตอบกลับช้าเกิน 10 วินาที (Timeout)
- ข้อมูล JSON ที่ส่งมาไม่ตรงกับที่ทางเราส่งไป
- ตอบกลับ HTTP Status Code อื่นที่ไม่ใช่ 2XX OK

ระบบจะถือว่า "การส่งล้มเหลว" และจะนำข้อมูลดังกล่าวเข้าสู่คิวส่งซ้ำ โดยอัตโนมัติ จำนวนการส่งซ้ำสูงสุด 3 ครั้ง

ตามตารางด้านล่างนี้:

| การส่งซ้ำ  | ระยะเวลารอ                            |
|------------|---------------------------------------|
| ครั้งที่ 1 | รอ 3 นาที หลังจากส่งครั้งแรกไม่สำเร็จ |
| ครั้งที่ 2 | รอ 5 นาที นับจากครั้งที่ 1            |
| ครั้งที่ 3 | รอ 10 นาที นับจากครั้งที่ 2           |

**\*\*หากส่งครบ 3 ครั้งแล้วยังไม่สำเร็จ ข้อมูลจะถูกบันทึกสถานะเป็น Dead Letter และส่งแจ้งเตือนไปยัง email ของท่านว่าระบบของท่านมีปัญหา ไม่สามารถส่งข้อมูลไปหาท่านได้ ทุกๆ 30 นาทีระบบจะทำการยิง dummy Test ไปหาท่านเพื่อเช็ควาระบบใช้งานได้หรือไม่ ถ้าระบบของท่านกลับมาทำงานได้ปกติ จะทำการส่งข้อมูลที่ท่านยังไม่ได้รับ\*\***

หมายเหตุ: อีเมลแจ้งเตือนจะถูกส่งก็ต่อเมื่อมีข้อมูลติดต่อกัน 3 ครั้ง (เข้า Dead Letter 3 ครั้งติดกัน โดยไม่มีครั้งไหนสำเร็จคั่นกลาง) สำหรับ Endpoint เดียวกันเท่านั้น — ไม่ใช่ทุกครั้งที่มิข้อมูล 1 รายการเข้า Dead Letter หลังจากตัดไฟแล้ว ข้อมูลใหม่ที่เข้ามาจะถูกบันทึกเป็น Dead Letter ทันทีโดยไม่มีการส่งซ้ำ 3 ครั้งอีก จนกว่า Endpoint จะฟื้นตัว